

Final Project Proposal

Derek Yu

October 26, 2025

AAE590 – Fundamentals of Estimation

Dr. Keith LeGrand

1. Proposed Problem Description

Drones need to operate in contested, jammed environments. The proposed project will investigate UAV navigation using terrain measurements (LiDAR measurements) and a digital elevation model (DEM). A simulated UAV's LLA, velocity, and attitude will be attempted to be estimated. The simulated UAV will be flying through waypoints (some combination of straights and turns) with constant velocity so that the dynamics will be linear. The elevation measurements will be generated using the simulated aircraft's states (truth), the DEM and additional noise/bias from a spec sheet of a commonly used LiDAR sensor used for terrain navigation. The measurements may be 1D or multi-dimensional and will be highly non-linear since Earth topology varies considerably, almost randomly.

2. Proposed Methodology

A particle filter will be used to estimate the location (and potentially orientation) of the simulated UAV in 3D space (ode45) flying in an area around the Grand Canyon. A simulated UAV flying some waypoints with truth states (LLA, velocity, attitude) will be used to generate noisy LiDAR measurements and will be used as reference for performance analysis of the particle filter. Real DEM data will be used from either [OpenTopography.org](https://opentopography.org) or <https://earthexplorer.usgs.gov>. This data will be used as measurements for computing residuals and weights for the particles. The code will be done either in Matlab or Python and potentially ported into C++ for performance speed comparison.

3. Anticipated Challenges

The implementation of a particle filter is likely the most challenging portion of this project since I have little to no experience implementing and testing one, but I am confident that with enough time and working through all the bugs, one can be implemented. Terrain navigation is not a new concept and has been proven to be feasible method of navigation.

Time permitting, the particle filter can be upgraded to an Adaptive Monte Carlo Localization particle filter such that the number of monte carlo draws computed on each

cycle reduces in reduce net computations required. The drawback to this method is that if the number of Monte Carlo draws are reduced too much, we may converge on an incorrect or poor state estimate.

4. Expected Results

As stated previously, terrain navigation is not a new concept and has been proven. This [paper](#) has claimed that their terrain navigation based underwater autonomous vehicle had no greater than 5m error. The 2020 Mars landing used terrain navigation with [reported 40 meters](#) accuracy which took up to “10 seconds from taking photos of landmarks and matching to a preloaded map to obtain the spacecraft position.” The key result of this project is that our particle filter should probably yield accuracy of dozens of meters roughly error relative to the truth simulated aircraft.

As mentioned previously, performance will primarily be measured as error between estimated vehicle states vs truth. It might also be helpful to look at our uncertainty estimate. A 3D trajectory plot of the estimated vehicle state and truth would be useful to look at. A video of the particle sampling around the flight path and convergence over time would also be great visualization to have for understanding what the filter is doing and for debugging purposes.

Given that there is sufficient time, another location like an urban city with more distinct features could be tested to see how terrain navigation would work. In theory, it could work better given that having more terrain details could lead to faster convergence, analogous to matching two keys together from a pile of keys than matching to match two smooth pebbles from a pile of smooth pebbles.

Given that storing large DEM's can be expensive in memory, analysis can be done where a range of DEM particles are run through several simulations and the performance (difference between estimated state and truth) is analyzed.

Additionally, the Matlab or Python implementation can be ported into C++ and a runtime analysis can be done to compare speed at which each script can complete running to get a rough order of magnitude of how much faster a compiled language is vs. interpreted.